Learning Predictive Autoscaling Policies for Cloud-hosted Microservices Using Trace-driven Modeling

Muhammad Abdullah†, Waheed Iqbal†, Abdelkarim Erradi‡, Faisal Bukhari†
†Punjab University College of Information Technology,
University of the Punjab, Lahore, Pakistan.
‡Department of Computer Science and Engineering,
College of Engineering, Qatar University, Doha, Qatar.
Email:{muhammad.abdullah, waheed.iqbal, faisal.bukhari}@pucit.edu.pk, erradi@qu.edu.qa

***Abstract*—Autoscaling methods are important to ensure response time guarantees for cloud-hosted microservices. Most of the existing state-of-the-art autoscaling methods use rule-based reactive policies with static thresholds defined either on monitored resource consumption metrics such as CPU and memory utilization or application-level metrics such as the response time. However, it is challenging to determine the most appropriate threshold values to minimize resource consumption and performance violations. Whereas, predictive autoscaling methods can help to address these challenges. These methods require considerable time to collect sufficient performance traces representing different resource provisioning possibilities for a target infrastructure to train a useful predictive autoscaling model. In this paper, we tackle this problem by proposing a system that models the response time of microservices through stress testing and then uses a trace-driven simulation to learn a predictive autoscaling model for satisfying response time requirements automatically. The proposed solution reduces the need for collecting performance traces to learn a predictive autoscaling model. Our experimental evaluation on AWS cloud using a microservice under realistic dynamic workloads validates the proposed solution. The validation results show excellent performance to satisfy the response time requirement with only 4.5% extra cost for using the proposed autoscaling method compared to the reactive autoscaling method.**

***Index Terms*—Cloud computing; Predictive autoscaling; SLO violations; Microservices; Response time guarantees.**

I. Introduction

Autoscaling is an important feature of Infrastructure-as-a-Service (IaaS) cloud that enables the users to dynamically allocate or remove computing resources to hosted applications to meet specific Service Level Objectives (SLOs) particularly response time SLOs [1]. However, satisfying a particular time of response SLO for the cloud-hosted application using minimal resources is challenging because identifying appropriate resource provisioning policies for dynamic workloads require considerable time to collect sufficient application performance traces for training the prediction models. Moreover, different resource provisioning possibilities are needed to test on target infrastructure under different dynamic workloads for learning useful provisioning policy model to satisfy performance requirements with minimal cost.

Recent advancements in service-oriented computing introduced the microservices architecture to enable building applications as a composition of independent and loosely coupled microservices that interact with each other to serve the incoming requests. The microservices offer fine-grained specific application features which demand particular hardware resources to serve large concurrent user requests. Nowadays, many applications especially Web API, mobile applications backends, workflow applications are based on the microservices architecture. This architecture is rapidly gaining traction due as it enables code reusability, eases maintainability, and reduces complexity by refactoring the application into microservices that can be developed and managed independently.

An automatic scaling method for microservices is required to meet the desired response time SLOs. Most of the existing autoscaling methods are based on reactive rules which are triggered based on specific events and resource utilization thresholds. For example, a reactive autoscaling method can increase the allocated resources for the service whenever CPU, memory, or network utilization reaches a specific level [2]. However, rule-based autoscaling methods cannot eliminate SLO violations as they are slow to respond to sudden workload bursts. Additionally, it is challenging to determine the optimal resource utilization thresholds to trigger the scaling operations due to exponentially large possibilities [3].

Alternatively, predictive autoscaling methods offer a better solution to minimize SLO violations for cloud-hosted services by proactively allocating or removing resources based on provisioning policies to satisfy the resource requirements of the forecasted workload. Most of the existing predictive autoscaling methods [4], [5] are either using analytical modeling [6] or reinforcement learning [7]. However, these methods require the knowledge of the application and sufficient time to run the application on the target infrastructure to collect training data for building predictive autoscaling models. In this paper, we address the problem of learning resource provisioning policies with minimal time, cost, and without any application knowledge. Our proposed approach consists of three steps. First, we learn the microservice response time model using a black-box

2330-2186/19/$31.00 ©2019 IEEE
DOI 10.1109/CloudCom.2019.00028

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

stress testing approach. Second, we emulate the VMs hosting the microservice through trace-driven simulation and perform autoscaling using a reactive policy to collect a training dataset for learning the predictive autoscaling policies. Finally, we train a regression model to predict the resources (number of VMs) required to satisfy a specific response time SLO for the forecasted workload. We investigated different commonly used machine learning methods to learn the resource provisioning policies and the workload forecasting methods for the sample benchmark microservice using two realistic and two synthetic workloads. Our experimental evaluation on AWS cloud validates the proposed solution. The main contributions of this paper include:

i Propose a technique for microservice response time modeling using stress testing.
ii Employ a trace-driven simulation to generate performance traces for learning predictive autoscaling policies model.
iii Study the effect of different workload forecasting methods and observation window sizes using two synthetic and two real workloads traces.
iv Propose a resource provisioning policies model using trace-driven simulation data and forecasted workload.
v Experimental evaluation of the proposed solution using multiple workloads and a benchmark microservice.
vi Validate the proposed solution over the AWS public cloud infrastructure.

The rest of the paper is organized as follows. The related work is presented in Section II. We briefly discuss the overall proposed system in Section III. We explain the proposed system methodology for learning predictive autoscaling resource provisioning policies in Section IV. The experimental evaluation is presented in Section V. Finally, the conclusions and future work are discussed in Section VI.

II. Related Work

There have been several efforts to maintain the performance of cloud-hosted applications using autoscaling methods. Lorido-Botran et al. [8] presents an extensive survey of different autoscaling methods for cloud-hosted applications. For example, Krieger et al. [9] use horizontal scaling method with a reactive strategy for bioinformatics and biomedical applications hosted on the cloud to improve the application performance. Recently, Liu et al. [10] present a fuzzy logic based approach to automatically adjust the thresholds and cluster size for reactive autoscaling method. Abdullah et al. [11] use reactive autoscaling to enhance the performance of the microservices-based application. Jindal [12] use performance modeling for microservices-based applications hosted on the cloud.

Recently, many researchers are working on predictive autoscaling methods. For example, Radhika et al. [13] proposed a predictive autoscaling method by using the autoregressive integrated moving average (ARIMA) and recurrent neural network. The authors predict the future workload using CPU and memory utilization to autoscale the application. Moreover, several works such as [7] proposed a reinforcement learning based solution to learn predictive autoscaling policies. However, such methods requires the application to run in the exploration phase for a significant time to gather sufficient performance traces.

Few efforts have been done to build autoscaling methods for microservices. For example, Alipour et al. [14] use machine learning to predict and provision resources required for microservices. The proposed method collects the resource demands and workload metrics for a specific workload and then uses linear regression and multiclass classification to predict and dynamically allocate resources. López et al. [15] use a mathematical model for autoscaling of microservices-based applications by identifying an appropriate number of replicas needed to maintain the desired application performance. The proposed method uses performance metrics collected using different combinations of replicas to learn the the resource provisioning policies.

Most of the existing work did not consider different workloads and require the microservices to run on the target infrastructures for a significant time to collect enough performance traces to build the predictive autoscaling model. Whereas, our work focuses on developing an automatic system to learn workload-specific predictive resource provisioning policies for microservices without requiring the application to run on the target infrastructure.

III. Proposed System Overview

The overall proposed methodology to automatically learn predictive resource provisioning policies is illustrated in Figure 1. The system workflow consists of the following steps:

1) The synthetic workload generator emulates concurrent user requests to stress test the microservice deployed on an Amazon EC2 instance. The workload generates requests in a step-up fashion until the microservice response time saturates. The system collects the performance traces by logging the number of requests and 95th percentile of response time during the stress test for one minute time intervals. We explain this in Section IV-A.
2) The performance traces are used to model the microservice response time. We found that an exponential curve appropriately captures the response time behavior of the benchmark microservice.
3) After learning the response time model, we perform a trace-driven simulation to generate the training dataset using a reactive autoscaling method for a linearly increasing workload to explore different system and resource allocation states. During the simulation, we continuously collect the performance traces that contain the number of requests, 95th percentile of response time (RT), and the number of VMs used to satisfied a specific response time SLO (τ_{slo}) for each one minute time interval. We explain this in Section IV-B.
4) The collected performance traces during the simulated autoscaling experiment are used to learn the resource provisioning policy model using a regression model which predicts the number of VMs required to satisfy a specific response time SLO for a given workload. We explain this in Section IV-C.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

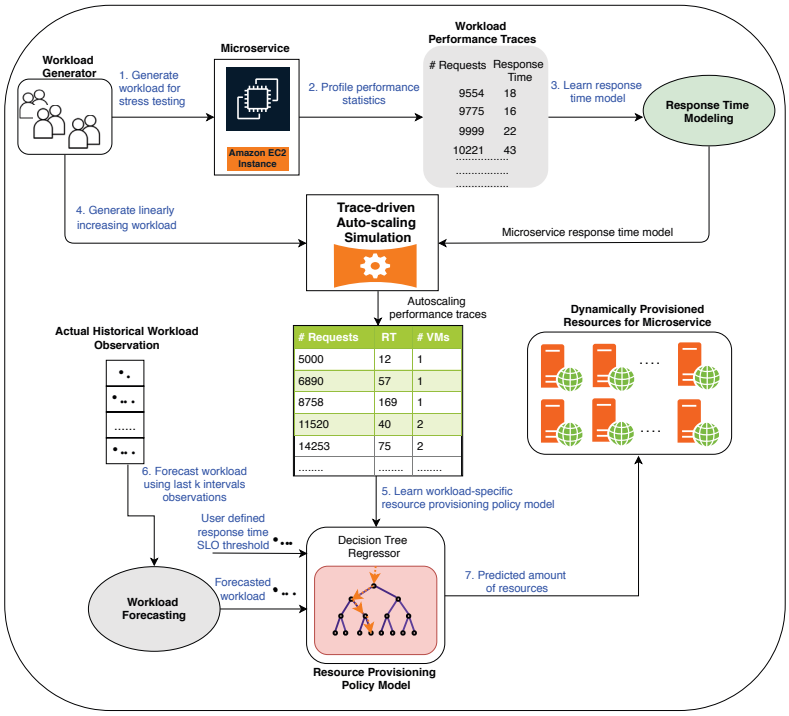


Fig. 1: Proposed system design to learn predictive autoscaling policies model for satisfying the response time SLO.

5) Once the resource provisioning policies model is trained, the system uses actual workload observations for the last k intervals to forecast the workload for the future interval. The forecasted workload is used to predict the number of VMs required to satisfy the response time SLO. We explain this in Section IV-D.
6) Finally, the system dynamically adjusts the resources allocated to the microservice according to the predicted resources for satisfying the response time requirements of the forecasted workload.

IV. Proposed System Methodology

A. Stress Testing for Response Time Modeling

We consider a quadratic equation solver as a benchmark microservice to emulate a CPU bound application. The quadratic equation microservice is written in PHP and accepts three parameters as the coefficient of the quadratic equation to find the roots. The coefficients are randomly selected between 1 to $1,000$ for each request of the synthetic workload. The output for each request is an HTML page showing the roots of the quadratic equation.

We deployed the benchmark microservice on an Amazon EC2 instance of type `c5.large` and performed a stress test by generating synthetic workload using httperf. The workload generates concurrent user requests in a step-up fashion to dramatically saturate the response time of the microservice. The system collects the performance traces during the stress test by logging the number of requests and 95th percentile of response time for a constant time interval consisting of one minute.

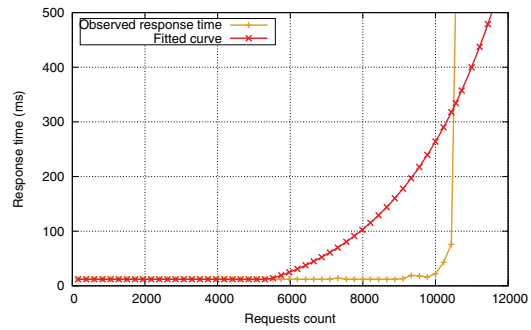


Fig. 2: Observed and fitted response time using the data collected during stress test of the benchmark application.

After the stress test, we model the response time using a curve fitting technique. Figure 2 shows the observed response time and fitted curve using the data collected during stress test of the benchmark application. We explored different curve fitting techniques and found the exponential curve with parameters $a = 8.24018$, $b = -0.0003632$, and $c = -47.27967$ appropriately generalized the response time of the benchmark application on a varying number of user requests using the following equation:

$$r_t = ae^{-bn_t} + c, \tag{1}$$

where r_t is the response time for the given number of requests n_t of time interval t.

B. Trace-driven Simulation for Dataset Collection

We performed a trace-driven simulation to collect a dataset for learning predictive the autoscaling policies model. In the

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

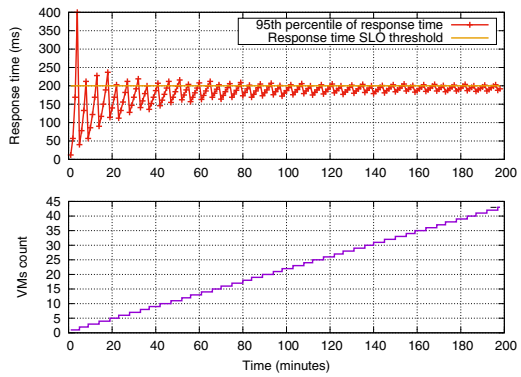


Fig. 3: Trace-driven autoscaling simulation results having 95th percentile of response time and dynamic allocation of VMs.

simulation, the VMs hosting the benchmark application are emulated using the response time model learned through stress testing the application on AWS cloud. Whereas the autoscaling of the microservice is enabled using a reactive policy for a linearly increasing workload. The reactive policy dynamically adds one VM instance whenever 95th percentile of the microservice response time is greater than 200 milliseconds. The linearly increasing workload begins generating 5000 concurrent requests and then increasing by 2000 requests every minute for a total 198 minutes duration.

During the simulation, we log the total number of requests, the microservice response time, and the VMs count for each time interval. Figure 3 shows the 95th percentile of response time and the VMs count during the simulation.

Algorithm 1 shows the reactive autoscaling method. We use an implementation similar to [1] by extending it for scale-out and scale-in on the basis of response time. The algorithm scales-out the allocated resources by adding one VM whenever 95th percentile of response time is greater than the user define SLO threshold τ_{slo} and scale-in the allocated resources whenever three consecutive intervals show 95th percentile of the response time less than $\frac{\tau_{slo}}{2}$.

C. Learning Predictive Autoscaling Policies Model

To learn a predictive autoscaling policies model, we used the data collected through trace-driven simulation and linearly increasing workload. We used the number of requests and 95th the percentile of response time as input to learn the number of VMs required to satisfy a specific response time SLO. In the dataset, all intervals showing SLO violations are discarded to learn the model by only using the data satisfying SLO. To evaluate the different regression methods, we used 80% of the data for training and 20% data for testing. We evaluated various state-of-the-art regression methods including Elastic Net (EN), Linear Regression (LR), Polynomial Regression (PR), Decision Tree Regressor (DTR), and Random Decision Forest (RDF).

Figure 4 shows normalized MSE Mean Square Error (MSE) for the regression methods used to learn the model for resource provisioning policies. We found that DTR outperform all

Algorithm 1: Reactive Autoscaling Method.

Input: Monitoring interval duration ($\varkappa$), response time SLO threshold (τ_{slo}), allocated set of resources ($\mathbb{V}$) to the microservice.
Output: Updated set of allocated resources $\mathbb{V}'$

```
while true do
    Wait for ϰ seconds and monitor the microservice logs
    r_t ← Extract 95^th percentile of response time from
      the logs for the current interval t
    if r_t > τ_slo (SLO Violation) then
        V+        ▷ Scale-out the microservice resources by
          adding one VM
    else if r_t & r_{t-1} & r_{t-2} < τ_slo/2 then
        V−        ▷ Scale-in microservice resources by
          removing one VM.
    end
    t ← t + 1
end
```

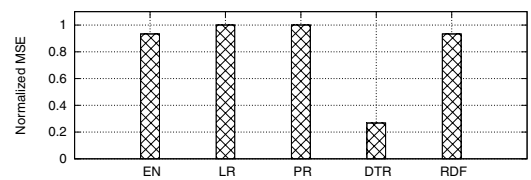


Fig. 4: Prediction error for Elastic Net (EN), Linear Regression (LR), Polynomial Regression (PR), Decision Tree Regression (DTR), and Random Decision Forest(RDF) on the test dataset for learning autoscaling policies model.

methods to minimize the error on testing data. Equation 2 shows our predictive autoscaling policies model:

$$M : w \times r \rightarrow c, \tag{2}$$

where M represents the trained model which can predict the required number of VMs c for a given number of requests w and a desired response time r.

Algorithm 2 shows the use of the trained autoscaling policies model for provisioning resources to the microservice. The algorithm accepts an application monitoring interval ($\varkappa$), a user-defined response time SLO threshold (τ_{slo}), a window size for workload forecasting (k), the trained predictive autoscaling policy model(M) and the current allocated resources ($\mathbb{V}$) to the microservice. In each iteration, it waits for $\varkappa$ seconds to collect the application performance traces and then forecasts $\widehat{w}$ the workload for the next interval $t+1$ using the observed workload for the last k intervals. After the workload forecasting, the predictive autoscaling policies model M is used to predict the number of VMs c required by the forecasted workload $\widehat{w}$ to satisfy the given response time SLO threshold τ_{slo}. The predicted number of VMs is used to adjust the set of allocated resource $\mathbb{V}$.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Predictive Autoscaling Method.

Input: Monitoring interval duration ($\varkappa$), Response time SLO threshold (τ_{slo}), Window size for workload forecasting (k), Predictive autoscaling model M , Allocated resources ($\mathbb{V}$) to the microservice.

Output: Updated allocated resources $\mathbb{V}'$

```
while true do
    Wait for ϰ seconds
    ŵ ← forecast the workload using a_t, a_{t-1}, ···, a_k observed workload.
    c ← M(ŵ, τ_slo)        ▷ Predict the resources count for the forecasted workload to satisfy the τ_slo.
    if |V| < c then
        V+        ▷ Scale-out the microservice resources by adding c − |V| VMs.
    else if |V| > c then
        V−        ▷ Scale-in the microservice resources by removing |V| − c VMs.
    end
    t ← t + 1
end
```

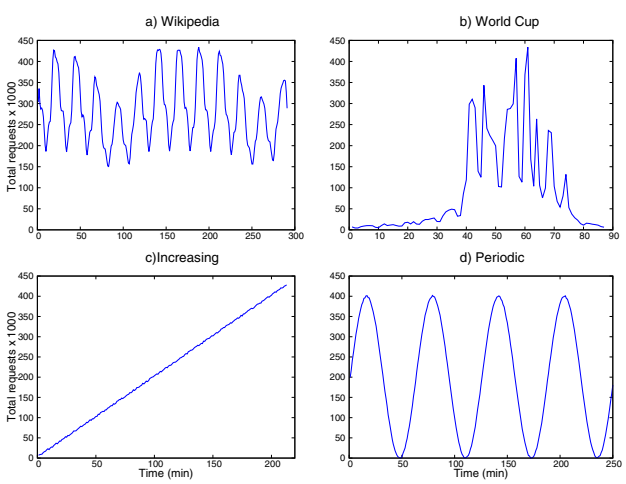


Fig. 5: Workloads used in the experimental evaluation.

D. Workload Generation and Forecasting

To evaluate the effectiveness of the predictive autoscaling model, we used four different types of workload namely Wikipedia, World cup, linearly increasing, and periodic workloads. The Wikipedia [16] and World cup [17] workloads are extracted from real traces available publicly and represent a real workload. Whereas, the linearly increasing and the periodic workloads are used to synthetically emulate commonly observed microservices workloads. Figure 5 shows the workloads used in the experimental evaluation.

To emulate linearly increasing workload, we consider 215 time intervals where each time interval consists of one minute. The workload begins generating a fix number of initial requests, $w_i(t_o)$ and then keep increasing the number of requests using a fix value, Δw_i. At a given time interval t, the number of requests in t-th interval $w_i(t)$ is generated by

$$w_i(t) = w_i(t_o) + (t-1)\Delta w_i. \tag{3}$$

The periodic workload is commonly observed by all application mainly as users access the application at specific times of the day. To generate a periodic workload, we used a sine function for a time period of p, an amplitude w_m and a bias of $w_p(t_o)$. At a given time interval t, the number of requests in t-th interval $w_p(t)$ is generated using:

$$w_p(t) = w_p(t_o) + w_m \sin(\frac{2t\pi}{p}). \tag{4}$$

In our predictive autoscaling method, we use the forecasted workload to predict the number of VMs required to handle the incoming workload. To forecast the workload for the next interval, the data of the last k time intervals are used to train a workload forecasting model. We evaluated various state-of-the-art forecasting method including Linear Regression (LR), XGBoost regression (XGB), Elastic Net (EN) and Support Vector Regression (SVR) by using 10, 20, 30, 40, and 50 for k to investigate the effect of different window sizes. Figure 6 shows the normalized RMSE (rooted mean square error) for different forecasting methods, workloads, and window sizes. A forecasting method with minimal RMSE is considered the best choice. For Wikipedia workload, XGB with window size 20 yield the best performance. For World cup, EN with window size 10 outperforms other forecasting methods. Whereas for increasing and period workloads LR with window size 40 and XGB with window size 10 yield forecasting with minimal error.

In our experimental evaluation, we used the best algorithm and window size identified for each workload to forecast the future workload.

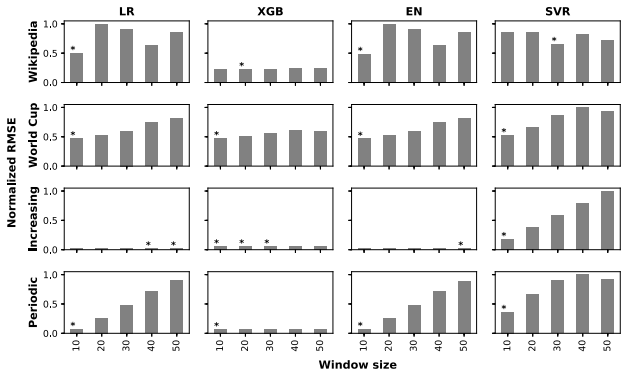


Fig. 6: Workload forecasting error for using different forecasting methods, workloads, and window sizes.

V. Experimental Evaluation

We evaluate the proposed system through trace-driven simulation for two synthetic and two real workloads as explained in Section IV-D and compare it with a baseline reactive autoscaling approach, explained in Algorithm 1. We validate the proposed predictive autoscaling method by performing

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

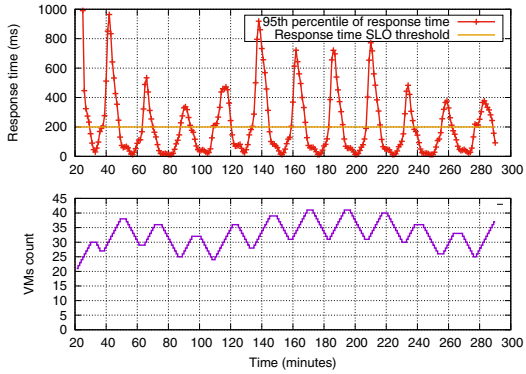


Fig. 7: Response time and adaptive allocation of VMs using reactive autoscaling method for Wikipedia workload.

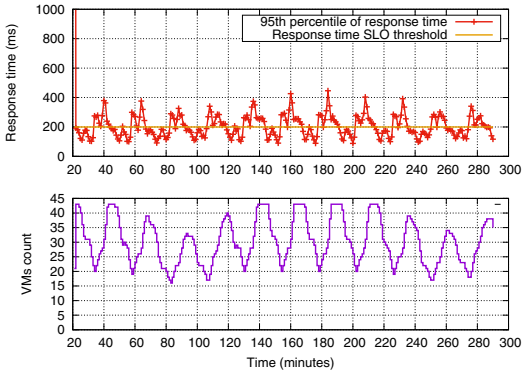


Fig. 8: Response time and adaptive allocation of VMs using the proposed predictive autoscaling method for Wikipedia workload.

experiments on Amazon Web Services (AWS) public cloud infrastructure for Wikipedia workload.

In our experimental evaluation, we compare the total processed requests, total SLO violations, and cost for used VMs for the proposed and baseline autoscaling methods. In simulations, the cost is computed by considering the per minute cost of AWS EC2 instance of type `c5.large`. Whereas, the cost for validation experiments showing the actual price of used VMs. In the following subsections, we present the results obtained in simulation and validation experiments.

A. Simulation-based Evaluation

Figure 7 shows the 95th percentile of response time and the adaptive allocation of VMs for the benchmark microservice using the reactive autoscaling method for Wikipedia workload. In the reactive autoscaling method, the system dynamically adds another VM whenever the 95th percentile of response time is greater than the response time SLO threshold (τ_{slo}), which is set to 200 milliseconds. The system scales-in the allocated resources whenever three consecutive intervals show 95th percentile of the response time less than $\frac{\tau_{slo}}{2}$. We observed SLO violations occurs but the reactive autoscaling method adaptively add more resources to bring the response time back to normal threshold repeatedly in this experiment due to the nature of the workload which grows and shrink over the time.

Figure 8 shows the 95th percentile of response time and adaptive allocation of VMs using the proposed predictive autoscaling method for Wikipedia workload. In this experiment, the system dynamically adjusts the allocated resources to the microservice according to the predictive number of VMs identified through the proposed system to satisfy a given response time SLO threshold. An autoscaling method with minimal SLO violations are considered the best choice to satisfy the response time requirements. The proposed autoscaling method significantly minimizes the SLO violations for Wikipedia workload as compared to the reactive autoscaling. Also, it yields slightly lower resources cost compared to the reactive method.

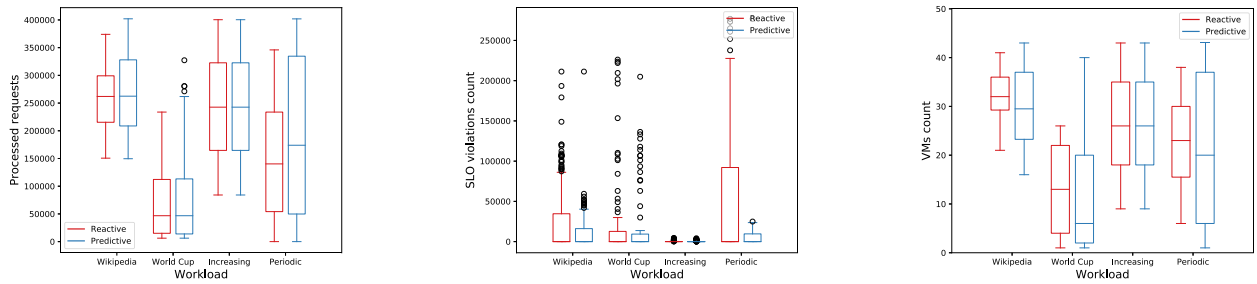
Table I summarize the performance comparison of the reactive and the proposed predictive autoscaling methods for each workload using trace-based simulation. We compute actual amount of the total processed requests, the total number of response time SLO violations, the percentage of processed requests, the percentage of SLO violations, and the total cost of used VMs. A method with a higher number of processed requests, a minimal number of response time SLO violations is considered better. The proposed predictive autoscaling method yields 4.6%, 9.0%, 0.06%, and 21.42% higher number of processed requests compared to the reactive autoscaling method for Wikipedia, World Cup, Increasing, and Periodic workloads respectively. The proposed predictive autoscaling method also shows minimum SLO violations for each workload compared to the reactive autoscaling method. We observed 57.1%, 24.4%, 35.2%, and 87.2% fewer SLO violations compared to the reactive autoscaling using the proposed method for Wikipedia, World Cup, Increasing, and Periodic workloads respectively.

We further compare and analyze the results through box plots, shown in Figure 9, for total processed requests, SLO violations, and number of VMs used during trace-driven simulation experiments for all four workloads. Figure 9a shows a box plot for total processed requests for Wikipedia, World Cup, Increasing, and Periodic workloads with reactive and the proposed predictive autoscaling methods. The proposed predictive autoscaling methods show a substantially higher number of total processed requests throughout the experiments using Wikipedia and Periodic workloads. Whereas World Cup and Increasing workloads show the comparable number of processed requests during the experiments for both the predictive and the reactive autoscaling methods. Figure 9b shows the box plot of SLOs violations observed using both the proposed predictive and reactive autoscaling methods for all of the workloads. The proposed method outperforms the baseline resource provisioning method by yielding significantly less SLOs during the experiments for Wikipedia, World Cup, and Periodic workloads. Whereas Increasing workloads show the comparable number of SLOs during the experiments for both the predictive and the reactive autoscaling methods. Figure 9c shows the box plot of VMs count using both the proposed

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Experimental results summary using trace-based simulation. Total number of processed requests, total SLO violations, percentage of processed requests, percentage of SLO violations, and total cost for all workloads using the reactive scaling and the proposed predictive autoscaling methods.

Workload	Autoscaling Method	Total Processed Requests	Total SLO Violations	Processed Requests (%)	SLO Violation (%)	Total Cost ($)
Wikipedia	Reactive	70,147,764	6,048,638	92.06	7.94	12.31
	Predictive	**73,606,313**	**2,590,089**	**96.60**	**3.40**	**11.44**
World Cup	Reactive	5,653,964	2,282,847	71.24	28.76	1.41
	Predictive	**6,218,981**	**1,717,830**	**78.36**	**21.64**	**1.25**
Increasing	Reactive	38,487,815	67,353	99.83	0.17	5.83
	Predictive	**38,510,997**	**44,171**	**99.89**	**0.11**	**5.84**
Periodic	Reactive	35,643,445	11,143,407	76.18	23.82	7.75
	Predictive	**45,364,019**	**1,422,833**	**96.96**	**3.04**	**7.11**



(a) Processed requests (b) SLO violations (c) VMs count

Fig. 9: Comparison of processed requests, SLO violations and VMs count during the simulation experiment.

predictive and reactive autoscaling methods for Wikipedia, World Cup, Increasing, and Periodic workloads.

B. Validation on AWS Cloud

To validate the proposed system on a real cloud infrastructure, we performed experiments on Amazon Web Services (AWS) cloud using the proposed autoscaling method and compared it with the reactive autoscaling method. The benchmark microservices application is deployed on AWS EC2 VM of type `c5.large` and Nginx is used as a load balancer. Nginx runs on a separate VM of type `c5.9xlarge` to avoid any saturation on the load balancer level. We generate Wikipedia workload by scaling it to a maximum of 35,000 requests per minute using httperf [18].

Figure 10 shows the 95th percentile of response time and adaptive allocation of VMs using the reactive autoscaling methods. The reactive autoscaling method identifies the need to scale-out or scale-in using the reactive approach as explained in Algorithm 1. We observed that the method appropriately adds more EC2 instances whenever the application response time grows higher than 200 (τ_{slo}) milliseconds and remove the EC2 instance whenever the application observed the response time lower than 100 ($\frac{\tau_{slo}}{2}$) milliseconds for three consecutive intervals.

Figure 11 shows the 95th percentile of response time and adaptive allocation of VMs using the proposed predictive autoscaling methods. The predictive autoscaling method identifies the need to scale-out or scale-in using the forecasted workload and resource provisioning policy model as explained in Algorithm 2. We observed that the method dynamically

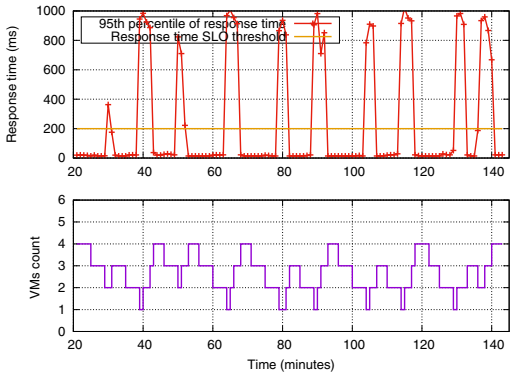


Fig. 10: Results of validation experiment showing response time and adaptive allocation of VMs during the experiment on AWS cloud using the proposed reactive autoscaling.

adjust the EC2 instances during the experiment. We found zero SLO violations using the proposed system to automatically provision the resource for satisfying the response time SLO.

We compare the total number of processed requests, SLO violations, and VMs cost statistics obtained using the proposed predictive autoscaling relative to the baseline reactive autoscaling results in Figure 12. The proposed autoscaling method on AWS cloud infrastructure yields 16.3% higher processed requests compared to the reactive autoscaling method. Whereas the proposed method shows zero SLO violations and therefore yielding 100% fewer SLO violations. However, the proposed solution produces 4.5% higher cost compared to the baseline.

Overall, the proposed predictive autoscaling method shows

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.

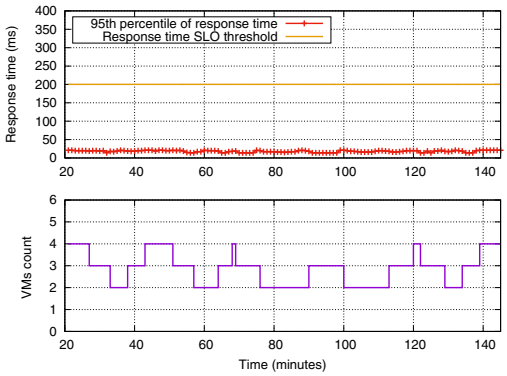


Fig. 11: Results of validation experiment showing response time and adaptive allocation of VMs during the experiment on AWS cloud using the proposed predictive autoscaling.

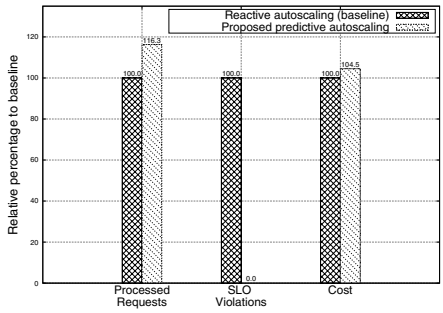


Fig. 12: Comparison of proposed reactive autoscaling results on AWS cloud relative to the baseline reactive autoscaling method.

an excellent performance by processing a higher number of requests, zero SLO violations, and comparable cost compared with the reactive autoscaling method. Moreover, the validation experiment shows that the trained autoscaling policies model through stress testing and trace-driven simulation is capable of adaptively scale-out and scale-in the microservice resources to indulge the SLO requirements for unseen and realistic workloads. Therefore, the proposed solution to automatically learn resource provisioning policies model can help to train predictive autoscalers for microservices efficiently by using minimal time and application performance traces from the target infrastructure.

VI. Conclusion and Future Work

Learning workload-specific predictive autoscaling policies is challenging because it requires the application to run on the target infrastructure for a considerable time to collect sufficient performance traces for a variety of workloads to train the predictive model. In this paper, we proposed and evaluated an efficient system to learn predictive autoscaling policies using the microservice response time model and trace-driven simulation to learn the predictive model in minimal time and without collecting large performance traces. Our experimental evaluation using simulations and real cloud infrastructure validates that the proposed system can efficiently learn the predictive autoscaling policies which adaptively manages the resources allocated to microservices to satisfy specific response time SLO requirements under different realistic and synthetic workloads.

Currently, we are extending our work by enabling feedback to the predictive autoscaling model for online retraining using operational performance traces. We also plan to automatically identify the best forecasting method and window sizes for fine tuning the proposed system.

Acknowledgement

This publication was made possible by NPRP grant # NPRP9-224-1-049 from the Qatar National Research Fund (a member of Qatar Foundation). The statements made herein are solely the responsibility of the authors.

References

[1] W. Iqbal, M. N. Dailey, D. Carrera, and P. Janecek, "Adaptive resource provisioning for read intensive multi-tier applications in the cloud," *Future Generation Computer Systems*, vol. 27, no. 6, 2011.

[2] W. Iqbal, M. Dailey, and D. Carrera, "SLA-driven adaptive resource management for web applications on a heterogeneous compute cloud," in *IEEE International Conference on Cloud Computing*. Springer, 2009.

[3] W. Iqbal, A. Erradi, and A. Mahmood, "Dynamic workload patterns prediction for proactive auto-scaling of web applications," *Journal of Network and Computer Applications*, vol. 124, 2018.

[4] W. Iqbal, A. Erradi, M. Abdullah, and A. Mahmood, "Predictive auto-scaling of multi-tier applications using performance varying cloud resources," *IEEE Transactions on Cloud Computing*, 2019.

[5] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: A taxonomy and survey," *ACM Computing Surveys*, vol. 51, no. 4, 2018.

[6] L. Aniello, S. Bonomi, F. Lombardi, A. Zelli, and R. Baldoni, "An architecture for automatic scaling of replicated services," in *Networked Systems*. Springer, 2014.

[7] W. Iqbal, M. N. Dailey, and D. Carrera, "Unsupervised learning of dynamic resource provisioning policies for cloud-hosted multitier web applications," *IEEE Systems Journal*, vol. 10, no. 4, 2016.

[8] T. Lorido-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *Journal of grid computing*, vol. 12, no. 4, 2014.

[9] M. T. Krieger, O. Torreno, O. Trelles, and D. Kranzlmüller, "Building an open source cloud environment with auto-scaling resources for executing bioinformatics and biomedical workflows," *Future Generation Computer Systems*, vol. 67, 2017.

[10] B. Liu, R. Buyya, and A. Nadjaran Toosi, "A fuzzy-based auto-scaler for web applications in cloud computing environments," in *Service-Oriented Computing*, C. Pahl, M. Vukovic, J. Yin, and Q. Yu, Eds. Springer International Publishing, 2018.

[11] M. Abdullah, W. Iqbal, and A. Erradi, "Unsupervised learning approach for web application auto-decomposition into microservices," *Journal of Systems and Software*, vol. 151, 2019.

[12] A. Jindal, V. Podolskiy, and M. Gerndt, "Performance modeling for cloud microservice applications," in *Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '19. New York, NY, USA: ACM, 2019.

[13] E. G. Radhika, G. S. Sadasivam, and J. Naomi, "An efficient predictive technique to autoscale the resources for web applications in private cloud," in *International Conference on Advances in Electrical, Electronics, Information, Communication and Bio-Informatics*, 2018.

[14] H. Alipour and Y. Liu, "Online machine learning for cloud resource provisioning of microservice backend systems," in *Big Data (Big Data), 2017 IEEE International Conference on*. IEEE, 2017.

[15] M. R. López and J. Spillner, "Towards quantifiable boundaries for elastic horizontal scaling of microservices," in *10th International Conference on Utility and Cloud Computing*. ACM, 2017.

[16] G. Urdaneta, G. Pierre, and M. van Steen, "Wikipedia workload analysis for decentralized hosting," *Comput. Netw.*, vol. 53, no. 11, Jul. 2009.

[17] M. Arlitt and T. Jin, "1998 world cup web site access logs," 1998.

[18] D. Mosberger and T. Jin, "httperf: A tool for measuring Web server performance," in *In First Workshop on Internet Server Performance*. ACM, 1998.

Authorized licensed use limited to: University of Greenwich. Downloaded on April 07,2026 at 15:57:35 UTC from IEEE Xplore. Restrictions apply.